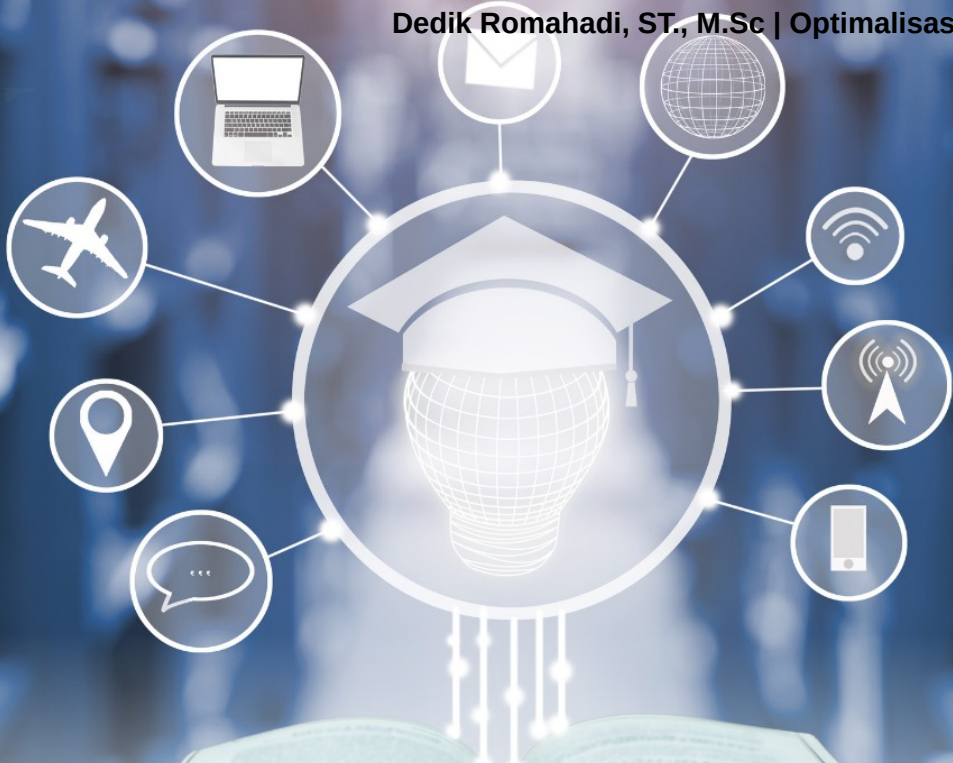




## MODUL PERKULIAHAN

# OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

## Modul 5

### Preprocessing Lanjutan: Transformasi

#### Abstrak

Pertemuan ini melanjutkan pembersihan data dengan teknik transformasi (log, Box-Cox, differencing, normalisasi) untuk

#### Sub - CPMK

Sub-CPMK 2.2 – Mampu merancang sistem otomasi menggunakan perangkat lunak simulasi

Disusun Oleh :  
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-5.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

## PENDAHULUAN

Data yang sudah “bersih” dari Pertemuan 4 belum tentu siap dimodelkan: masih ada tren, musiman, dan varians tidak konstan yang melanggar asumsi algoritma statistik dan machine learning. Pertemuan kelima ini membahas teknik transformasi untuk menstabilkan varians serta teknik dekomposisi untuk memisahkan komponen tren, musiman, dan residual, dirancang dan diuji menggunakan perangkat lunak simulasi (Python, NumPy, statsmodels) sebelum diterapkan pada sistem produksi. Gambar 1 menunjukkan posisi Pertemuan 5 dalam alur mata kuliah.



Gambar 1. Posisi Pertemuan 5 (Transformasi & Dekomposisi) dalam alur mata kuliah, menjembatani preprocessing dengan ekstraksi fitur pada Pertemuan 6-7.

Materi mencakup transformasi variance-stabilizing (log, Box-Cox), differencing untuk menghilangkan tren, normalisasi/standarisasi skala fitur, dekomposisi klasik dan STL, serta smoothing lanjutan. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan latihan komputasi.

### Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 2.2:** Mampu merancang sistem otomasi menggunakan perangkat lunak simulasi, yaitu merancang dan menguji pipeline transformasi serta dekomposisi deret waktu memakai Python sebelum diterapkan pada sistem monitoring produksi (CPMK 2).
- Setelah pertemuan ini mahasiswa dapat: menerapkan transformasi log, Box-Cox, differencing, dan normalisasi sesuai kebutuhan data; menjelaskan perbedaan dekomposisi aditif dan multiplikatif; menerapkan dekomposisi STL; serta memilih teknik smoothing lanjutan yang sesuai konteks.

## TRANSFORMASI DATA: LOG, BOX-COX, DIFFERENCING & NORMALISASI

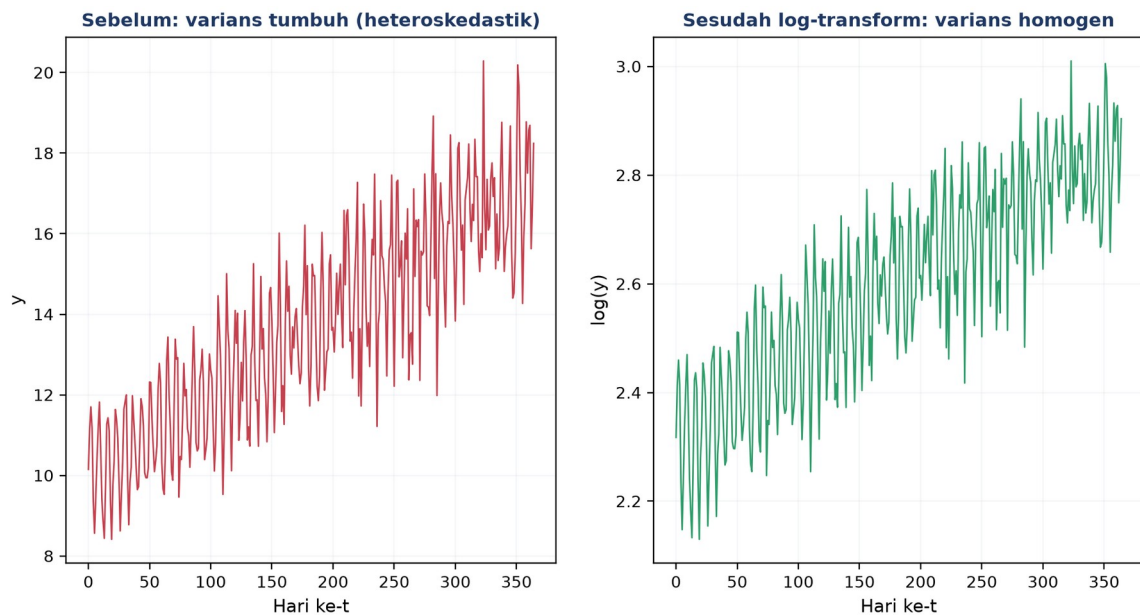
**Box-Cox:**  $y(\lambda) = (x\lambda - 1)/\lambda$  untuk  $\lambda \neq 0$ ;  $y(\lambda) = \log(x)$  untuk  $\lambda = 0$

- **Log Transform:**  $y' = \log(y+\epsilon)$ ; dipakai untuk data dengan pola varians yang membesar seiring mean (fan-shaped variance), mensyaratkan data positif.
- **Box-Cox ( $\lambda$  optimal):**  $\lambda$  optimal dicari melalui maximum likelihood estimation; nilai umum  $\lambda \in \{-1$  (reciprocal),  $0$  (log),  $0,5$  (sqrt),  $1$  (identity)}. Diimplementasikan via ``scipy.stats.boxcox``, mensyaratkan  $y > 0$  (pakai Yeo-Johnson bila data bisa nol/negatif).
- **Differencing:** orde-1 ( $\nabla x_t = x_t - x_{t-1}$ ) menghilangkan tren linear; orde-2 menghilangkan tren kuadratik; seasonal differencing ( $x_t - x_{t-s}$ ) menghilangkan pola musiman berperiode  $s$ .
- **Z-score & Min-Max:** Z-score untuk algoritma seperti k-NN, SVM, PCA; Min-Max untuk rentang 0-1 (jaringan saraf); keduanya harus di-fit hanya pada data train untuk menghindari data leakage.

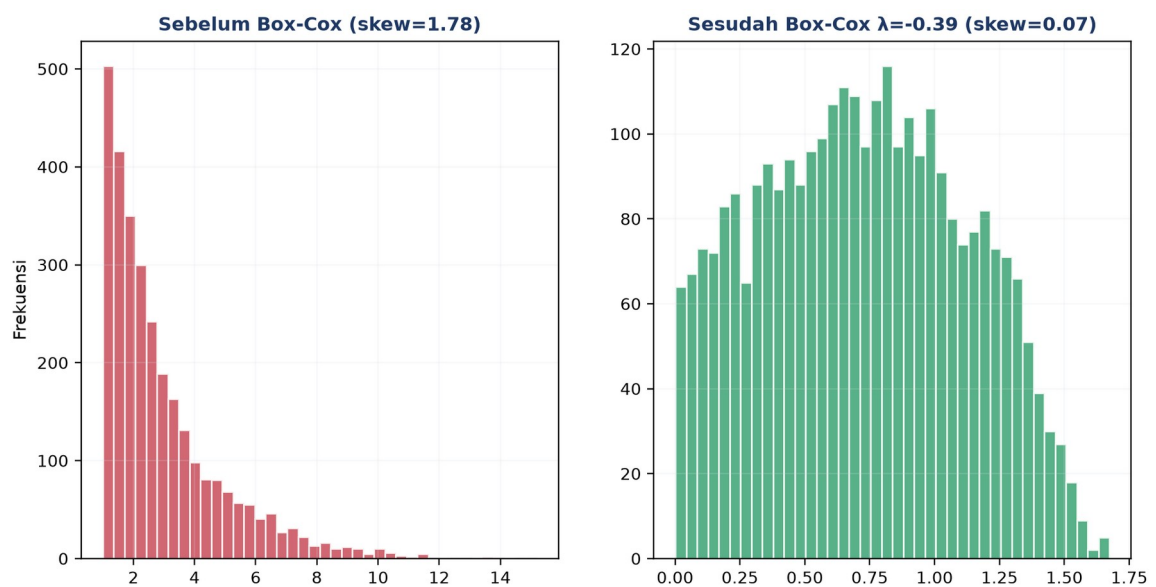
Tabel 1 merangkum enam teknik transformasi beserta kapan dipakai, sifat reversibilitas, dan catatan penting.

Tabel 1. Perbandingan enam teknik transformasi data: kapan dipakai, reversibilitas, dan catatan.

| Transformasi  | Kapan Dipakai                                         | Reversible?                            | Catatan                                                  |
|---------------|-------------------------------------------------------|----------------------------------------|----------------------------------------------------------|
| Log           | Varians tumbuh proporsional terhadap mean             | Ya, $y = \exp(y')$                     | Data harus positif; tambahkan epsilon kecil jika ada nol |
| Box-Cox       | Ingin mendekati normal; varians tidak stabil          | Ya, inverse tersedia                   | Hanya untuk $y > 0$ ; pakai Yeo-Johnson untuk $y \leq 0$ |
| Differencing  | Menghilangkan tren dan mencapai stasioneritas         | Ya, integrasi kumulatif                | Kurangi 1 observasi per orde; perbesar noise             |
| Z-score       | Algoritma sensitif jarak (k-NN, SVM, PCA, clustering) | Ya, $x = z \cdot \sigma + \mu$         | Output unbounded; sensitif outlier                       |
| Min-Max       | Jaringan saraf (aktivasi sigmoid/tanh)                | Ya, $x = z \cdot (\max - \min) + \min$ | Output 0-1 ketat; sangat sensitif outlier                |
| Robust Scaler | Data banyak outlier yang belum dibersihkan            | Ya                                     | Pakai median dan IQR; tahan outlier                      |



Gambar 2. Efek log-transform terhadap variance stabilization: sinyal asli dengan varians tumbuh (heteroskedastik) menjadi homogen setelah log-transform.

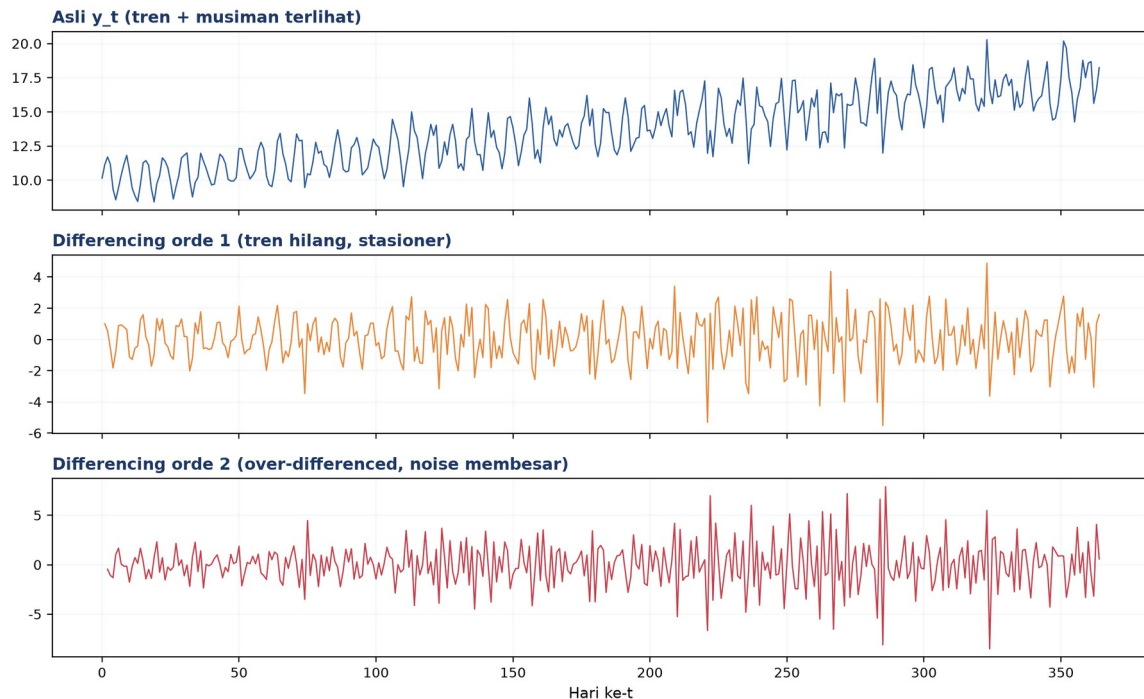


Gambar 3. Distribusi data sebelum dan sesudah Box-Cox transform; skewness turun dari 1,78 menjadi 0,07 mendekati distribusi normal.

**Info penting:** Aturan fit-transform harus dipatuhi untuk menghindari data leakage: scaler di-fit sekali pada data train (`scaler.fit(X_train)`), lalu dipakai berulang untuk transform data test (`scaler.transform(X_test)`), tidak pernah sebaliknya.

**Uji Stasioneritas dengan ADF Test:** Uji stasioneritas formal sebaiknya melengkapi inspeksi visual sebelum memutuskan orde differencing yang dipakai. Augmented Dickey-Fuller (ADF) test menguji hipotesis nol bahwa deret memiliki unit root (non-stasioner); p-value di bawah 0,05 mengindikasikan deret sudah cukup stasioner untuk dianalisis lebih lanjut. Praktik yang disarankan: mulai dari differencing orde 0 (data asli), jalankan ADF test,

bila masih non-stasioner naikan ke orde 1, uji lagi, dan seterusnya, lalu berhenti begitu p-value ADF signifikan, karena melanjutkan differencing lebih jauh dari yang diperlukan hanya memperbesar noise tanpa manfaat tambahan.



Gambar 4. Efek differencing: deret asli dengan tren dan musiman (atas), hasil differencing orde 1 yang stasioner (tengah), dan orde 2 yang over-differenced dengan noise membesar (bawah).

**Peringatan:** Lupa melakukan inverse transform saat menampilkan hasil prediksi menyebabkan error dilaporkan dalam skala yang salah, sebuah kesalahan praktis yang sering luput dari perhatian.

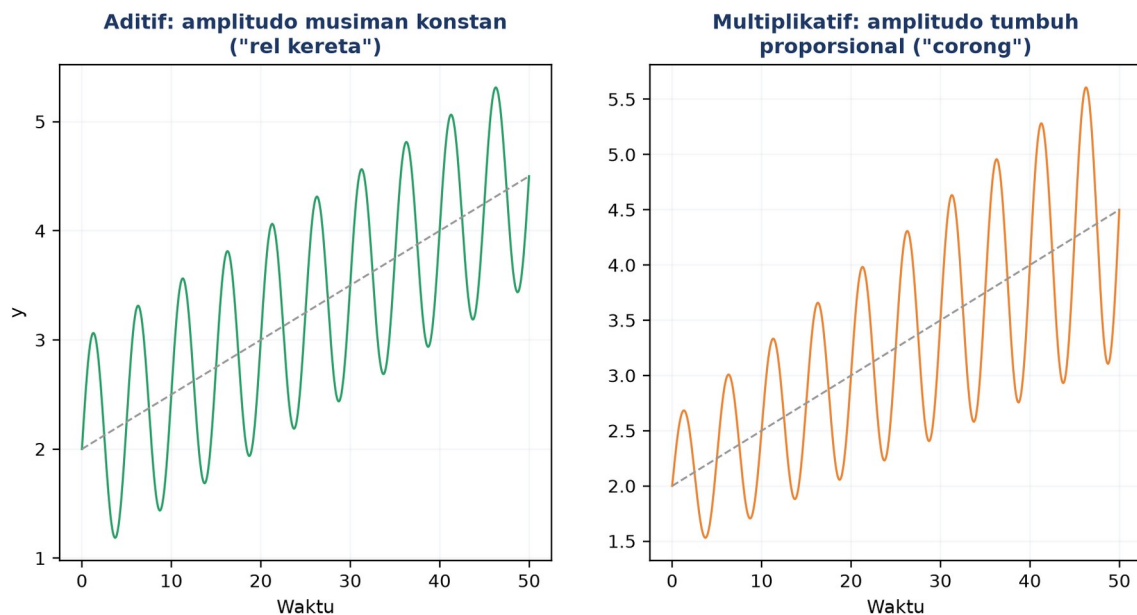
## DEKOMPOSISI DERET WAKTU: TREN, MUSIMAN, RESIDUAL

**Aditif:**  $y_t = T_t + S_t + R_t$

**Multiplikatif:**  $y_t = T_t \times S_t \times R_t$

- **Kapan Aditif?:** amplitudo pola musiman relatif konstan terlepas dari tren, contoh suhu pabrik harian berfluktuasi  $\pm 3^\circ\text{C}$  tanpa terpengaruh tren jangka panjang, mengikuti pola "rel kereta".
- **Kapan Multiplikatif?:** amplitudo pola musiman tumbuh proporsional dengan tren, contoh konsumsi energi pabrik naik 10% per tahun sehingga fluktuasi musimannya turut membesar, mengikuti pola "corong". Trik praktis:  $\log(y)$  mengubah pola multiplikatif menjadi aditif.
- **Komponen Tren:** dihitung via moving average centered dengan window sepanjang periode musiman ( $T_t = \text{MA}_k(y_t)$ ).

- **Komponen Musiman:** dihitung sebagai rata-rata data yang sudah di-detrend per posisi dalam periode ( $St = \text{rata-rata fase}(y-T)$ ).



Gambar 5. Perbandingan konseptual pola aditif (amplitudo musiman konstan, "rel kereta") dan multiplikatif (amplitudo tumbuh proporsional, "corong").

Diagnosa cepat aditif vs multiplikatif dapat dilakukan dengan mengamati plot: jika amplitudo musiman tetap konstan sepanjang deret waktu, pilih model aditif; jika amplitudo membesar seiring naiknya tren, pilih model multiplikatif.

## STL DECOMPOSITION & SMOOTHING LANJUTAN

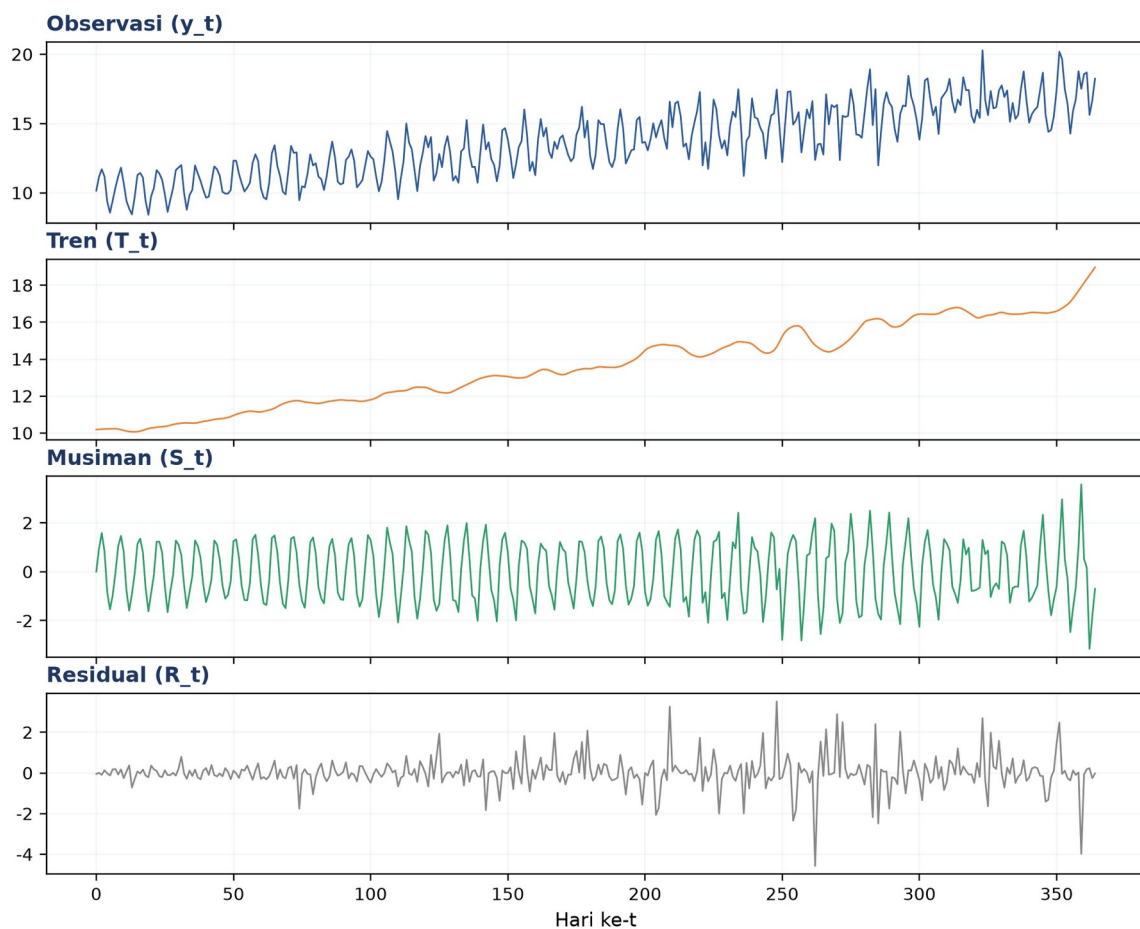
$$MA_k(y_t) = (1/k) \cdot \sum_{t-i} y_{t-i} \quad EMA_\alpha(y_t) = \alpha \cdot y_t + (1-\alpha) \cdot EMA_\alpha(y_{t-1})$$

- **STL, Keunggulan:** berbasis LOESS, mampu menangani musiman yang berubah secara bertahap, robust terhadap outlier; diimplementasikan via ``statsmodels.tsa.seasonal.STL``.
- **Moving Average (MA):** varian centered (tanpa lag, offline) vs trailing (causal, real-time); trade-off window besar (halus tetapi lag besar) versus window kecil (responsif tetapi berisik).
- **Exponential Moving Average (EMA):** populer untuk alerting real-time karena bersifat one-pass dan hanya butuh nilai sebelumnya.
- **Savitzky-Golay Filter:** fitting polinomial per window, menjaga puncak dan lembah sehingga cocok untuk data getaran yang bentuknya bermakna.

Tabel 2 merangkum lima metode smoothing/dekomposisi beserta keunggulan, kekurangan, dan kapan dipakai.

Tabel 2. Perbandingan lima metode smoothing/dekomposisi: keunggulan, kekurangan, dan kapan dipakai.

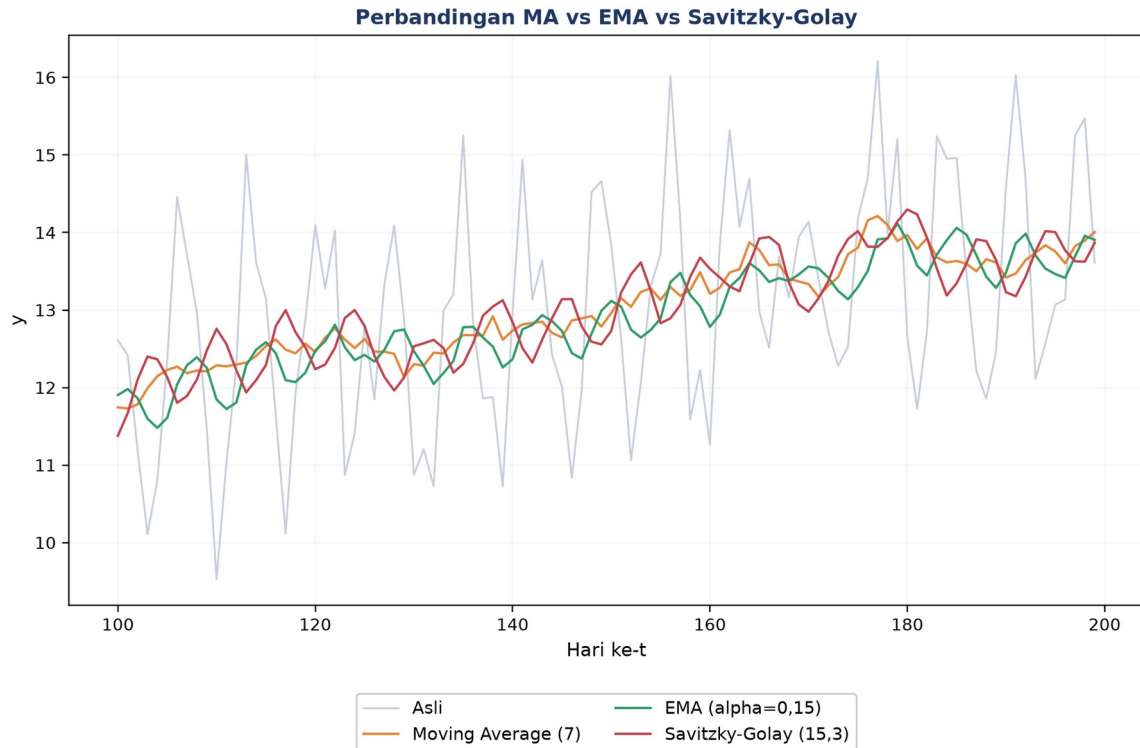
| Metode              | Keunggulan                             | Kekurangan                               | Kapan Dipakai                      |
|---------------------|----------------------------------------|------------------------------------------|------------------------------------|
| MA (Moving Average) | Sederhana, cepat, mudah diinterpretasi | Lag, smoothing sama untuk semua titik    | Dekomposisi klasik, baseline cepat |
| EMA (Exp. MA)       | Responsif, lag kecil, one-pass         | Sulit menentukan alpha optimal           | Alerting real-time, streaming      |
| STL                 | Seasonal dinamis, robust outlier       | Komputasi lebih mahal, butuh statsmodels | Forecasting industri serius        |
| Savitzky-Golay      | Jaga bentuk peak/trough                | Butuh tuning (w,p); asumsi data smooth   | Sinyal getaran, spektroskopi       |
| LOESS/LOWESS        | Non-parametrik, fleksibel, robust      | Sensitif terhadap bandwidth              | Eksplorasi tren non-linear         |



Gambar 6. Dekomposisi STL deret harian sintesis menjadi observasi, tren, musiman, dan residual, dengan periode musiman 7 hari.

**LOESS untuk Musiman yang Berubah:** LOESS/LOWESS layak dipertimbangkan sebagai alternatif STL saat pola musiman tidak konstan sepanjang waktu, misalnya pabrik yang mengubah pola shift kerja di pertengahan tahun sehingga periode musiman mingguan berubah karakternya. Parameter bandwidth pada LOESS mengontrol trade-off serupa dengan window size pada MA: bandwidth kecil mengikuti fluktuasi lokal secara ketat (risiko

overfitting ke noise), sedangkan bandwidth besar menghasilkan kurva tren yang lebih halus namun berpotensi kurang responsif terhadap perubahan struktural yang legitimate dalam data.



Gambar 7. Perbandingan Moving Average, EMA, dan Savitzky-Golay pada segmen data yang sama.

**Rekomendasi pipeline:** Pipeline preprocessing industrial yang direkomendasikan: (1) plot data untuk inspeksi awal; (2) terapkan log/Box-Cox bila diperlukan; (3) terapkan STL atau differencing; (4) periksa residual; (5) uji stasioneritas dengan ADF test; (6) simpan parameter transformasi untuk keperluan inverse.

**Peringatan:** Jebakan industri yang sering terjadi: lupa menyimpan parameter scaler sehingga terjadi data leakage;  $\log(0)$  menghasilkan NaN (solusinya pakai  $\log(x+1)$  atau Yeo-Johnson); periode musiman yang salah dimasukkan ke STL; serta over-differencing yang justru memperbesar noise.

## ANALOGI KEHIDUPAN SEHARI-HARI

- **Volume Konser & Skala Desibel:** telinga manusia bekerja secara logaritmik terhadap intensitas suara ( $\text{dB} = 10 \cdot \log(P/P_0)$ ), analog langsung dengan log-transform pada sinyal.
- **Saham & Return Harian:** log-return harga saham ( $r_t = \log(P_t/P_{t-1})$ ) menyetarakan saham dengan skala harga yang sangat berbeda agar bisa dibandingkan.

- **Penjualan Ritel & Weekend Spike:** dekomposisi memisahkan tren, musiman (lonjakan akhir pekan), dan residual; residual yang anomali bisa jadi sinyal aktivitas kompetitor.
- **Termometer Medis & Z-score:** Z-score suhu tubuh ( $z=(37,8-\mu)/\sigma$ ) dengan  $z>2$  dianggap abnormal, analog langsung dengan normalisasi multi-sensor.
- **Stok Gudang & Differencing:** differencing pada data stok kumulatif mengubahnya menjadi pola penerimaan harian yang lebih stabil dan mudah dianalisis.
- **Tinggi & Berat Badan:** tanpa scaling, algoritma seperti k-NN didominasi oleh fitur berskala besar (misal berat badan dalam gram vs tinggi dalam meter); Min-Max/Z-score menyetarakan kontribusi tiap fitur.

## APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Condition Monitoring Bearing:** pipeline  $\log(\text{RMS})$  lalu STL menghasilkan sinyal tren murni sebagai indikator kesehatan bearing, terpisah dari fluktuasi musiman harian.
- **Energy Monitoring Pabrik:** model  $E = T_y + S_w + S_d + R$  (tren tahunan + musiman mingguan + musiman harian + residual); residual yang anomali menjadi sinyal deteksi kebocoran energi.
- **Temperature Sensor Oven:** EMA dengan  $\alpha=0,05$  sebagai baseline, alert dipicu jika  $|y-\text{EMA}|>3\sigma$ .
- **SCADA Multi-Sensor:** normalisasi Z-score ( $z=(x-\mu_{\text{train}})/\sigma_{\text{train}}$ ) wajib dilakukan sebelum data multi-sensor dimasukkan ke Isolation Forest, One-Class SVM, atau autoencoder.

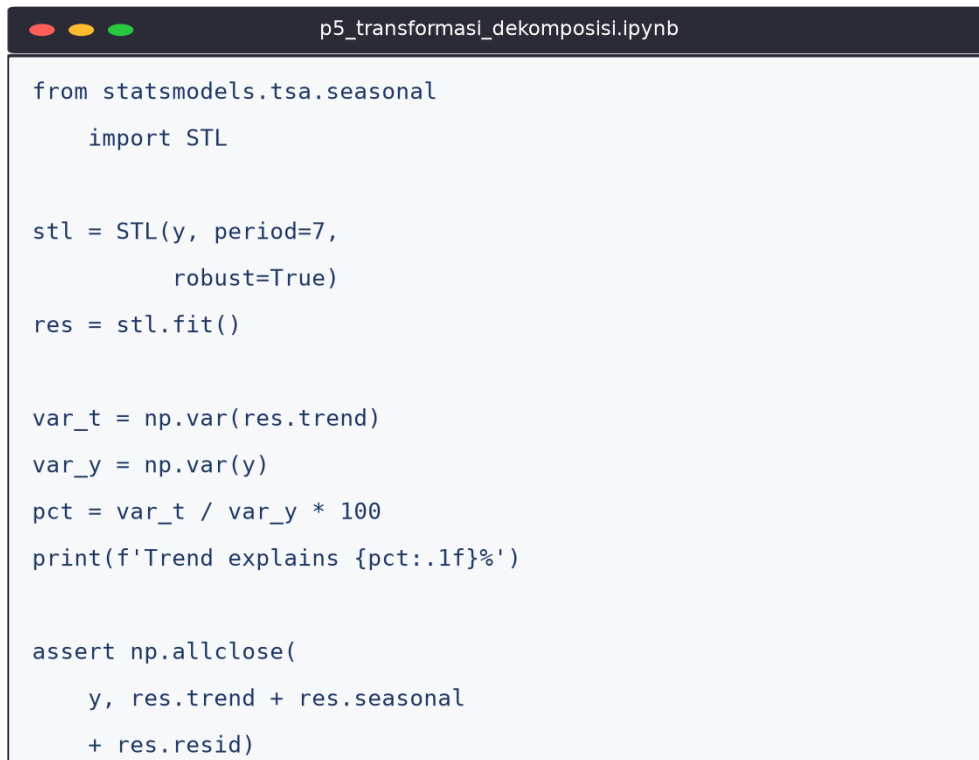
**Kaitan dengan pertemuan berikutnya:** Materi ini menjadi fondasi bagi Pertemuan 6 (fitur domain waktu yang sensitif terhadap varians) dan Pertemuan 7 (FFT, di mana musiman yang tidak didekomposisi dapat menghasilkan peak frekuensi palsu).

## IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada deret sintetis. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, statsmodels, matplotlib; notebook p5\_transformasi\_dekomposisi.ipynb. Gambar 8 menunjukkan cuplikan Cell 4 yang menjalankan dekomposisi STL dan memverifikasi rekonstruksinya.

**Sanity-Check Rekonstruksi STL:** Verifikasi rekonstruksi (memastikan observasi  $\approx$  trend + seasonal + residual dalam toleransi numerik kecil) merupakan langkah sanity-check yang

sering diabaikan tetapi penting: kegagalan rekonstruksi biasanya mengindikasikan periode musiman yang salah dimasukkan ke STL, atau data mengandung missing values yang belum ditangani sebelum dekomposisi dijalankan. Cell 4 pada notebook ini secara eksplisit menghitung selisih antara data asli dan hasil penjumlahan ketiga komponen, mencetak nilai maksimum residual rekonstruksi sebagai indikator kebenaran proses.



```
from statsmodels.tsa.seasonal
    import STL

stl = STL(y, period=7,
          robust=True)
res = stl.fit()

var_t = np.var(res.trend)
var_y = np.var(y)
pct = var_t / var_y * 100
print(f'Trend explains {pct:.1f}%')

assert np.allclose(
    y, res.trend + res.seasonal
    + res.resid)
```

Gambar 8. Cuplikan Cell 4: dekomposisi STL penuh dan verifikasi rekonstruksi  $y \approx \text{trend} + \text{seasonal} + \text{residual}$ .

- **Cell 1:** generate deret sintetis 365 hari (tren linear + musiman mingguan + noise heteroskedastik), plot dan statistik deskriptif.
- **Cell 2:** log transform + Z-score (`StandardScaler`, fit pada 80% data train) + Min-Max scaling, plot perbandingan 2x2.
- **Cell 3:** differencing orde 1 dan 2 (`np.diff`) plus ADF test (`statsmodels.tsa.stattools.adfuller`,  $p < 0,05$  berarti stasioner), plot 3-panel.
- **Cell 4:** dekomposisi STL penuh (period=7, robust=True), cetak persentase variance explained per komponen, plot 4-panel, verifikasi rekonstruksi.

## TUGAS PERTEMUAN 5

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.

### Distribusi Bobot Tugas Pertemuan 5



Gambar 9. Distribusi 50 poin Tugas Pertemuan 5 berdasarkan tipe soal.

### Bagian A: Pilihan Ganda (10 soal $\times$ 1 poin)

1. Kondisi stasioneritas pada deret waktu berarti...

- A. Data dikumpulkan pada interval waktu yang tetap
- B. Mean, variansi, dan struktur kovariansnya konstan sepanjang waktu
- C. Deretnya berbentuk garis lurus (linear)
- D. Semua nilai dalam deret adalah sama (tidak berfluktuasi)

2. Bila sebuah deret waktu menunjukkan varians yang tumbuh seiring mean (plot berbentuk corong), transformasi yang paling tepat adalah...

- A. Min-Max scaling ke  $[0, 1]$
- B. Differencing orde 2
- C. Log transform (atau Box-Cox)
- D. Menghapus semua observasi dengan nilai ekstrem

3. Pada transformasi Box-Cox dengan parameter  $\lambda=0$ , formula yang diterapkan setara dengan...

- A. Identitas, tidak ada transformasi
- B. Log natural,  $y' = \log(x)$
- C. Akar kuadrat,  $y' = \text{akar}(x)$
- D. Inverse,  $y' = 1/x$

4. Operasi differencing orde 1 ( $\nabla x_t = x_t - x_{t-1}$ ) pada sebuah deret waktu bertujuan utama untuk...

- A. Mengurangi jumlah observasi agar model lebih cepat dilatih
- B. Menghilangkan tren (linear) sehingga deret menjadi lebih stasioner
- C. Menyetarakan skala antar fitur sensor yang berbeda
- D. Menghilangkan outlier secara otomatis

5. Setelah menerapkan Z-score normalization dengan benar pada sebuah fitur, mean dan standar deviasi hasilnya menjadi...

- A. mean = 1, std = 0
- B. mean = 0, std = 1
- C. min = 0, max = 1
- D. mean = 0, std = 0

6. Dalam dekomposisi deret waktu, model multiplikatif ( $y_t = T_t \times S_t \times R_t$ ) paling tepat digunakan saat...

- A. Amplitudo pola musiman konstan terlepas dari tren
- B. Amplitudo pola musiman tumbuh proporsional dengan tren
- C. Deret tidak memiliki komponen musiman sama sekali
- D. Data bernilai negatif di banyak titik

7. Pada uji Augmented Dickey-Fuller (ADF) untuk stasioneritas, p-value < 0,05 artinya...

- A. Null hypothesis "non-stasioner" ditolak, deret dianggap stasioner
- B. Deret pasti memiliki tren linear yang kuat
- C. Deret pasti non-stasioner, butuh differencing tambahan
- D. Model tidak valid dan harus diulang dengan data baru

8. Dibandingkan dengan Moving Average, keunggulan utama Exponential Moving Average (EMA) untuk monitoring sensor real-time adalah...

- A. EMA selalu menghasilkan varians lebih kecil
- B. EMA lebih responsif (lag lebih kecil) dan bersifat one-pass, cocok untuk streaming
- C. EMA tidak perlu parameter apapun
- D. EMA menghasilkan output 0-1 secara otomatis

9. Praktik fit-transform yang benar untuk StandardScaler pada pipeline train/test split adalah...

- A. fit\_transform pada train dan fit\_transform pada test juga
- B. fit\_transform pada train, lalu transform saja pada test (pakai parameter train)
- C. Gabungkan train + test, lalu fit\_transform keseluruhan
- D. Hitung mu dan sigma per batch, masing-masing terpisah

10. Keunggulan utama STL (Seasonal-Trend decomposition using LOESS) dibandingkan dekomposisi klasik berbasis moving average adalah...

- A. STL selalu jauh lebih cepat dari moving average
- B. Komponen musiman boleh berubah bertahap sepanjang waktu dan lebih robust terhadap outlier
- C. STL tidak memerlukan penentuan periode musiman
- D. Hasil STL otomatis stasioner tanpa perlu differencing

### Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Dataset referensi (dipakai C1-C15, dibuat sekali di Jupyter):

```
import numpy as np, pandas as pd
np.random.seed(42)
N = 365
t = np.arange(N)
trend = 0.02 * t
seasonal = 1.5 * np.sin(2*np.pi*t/7)
noise = np.random.normal(0, 0.3, N)
y = 10 + trend + seasonal + noise
series = pd.Series(y, index=pd.date_range('2025-01-01', periods=N, freq='D'))
```

**C1.** Dari dataset referensi series, hitung mean sepanjang seluruh deret dengan `series.mean()`.

- 💡 **HINT:** Gunakan `series.mean()` untuk menghitung rata-rata deret tersebut.

**C2.** Hitung standar deviasi dari dataset referensi dengan `series.std()` (ingat pandas memakai default `ddof=1`).

- 💡 **HINT:** Gunakan `series.std()`; ingat pandas memakai default `ddof=1` (sample std).

**C3.** Terapkan log transform pada dataset referensi: `y_log = np.log(y)`. Hitung mean hasil log.

- 💡 **HINT:** Transformasi log dengan `np.log(y)`, lalu hitung rata-ratanya dengan `np.mean`.

**C4.** Terapkan Z-score normalization manual:  $z = (y - \text{mean})/\text{std}$ . Hitung standar deviasi dari `z` (harus mendekati 1,0).

- 💡 **HINT:** Standardisasi  $z = (y - \text{mean})/\text{std}$ , lalu hitung std dari `z` dengan `np.std(..., ddof=1)`.

**C5.** Terapkan Min-Max scaling manual ke rentang `[0,1]`:  $y_{\text{mm}} = (y - \text{min})/(\text{max} - \text{min})$ . Cetak nilai maksimum dari `y_mm`.

- 💡 **HINT:** Min-max scaling  $= (y - \text{min})/(\text{max} - \text{min})$ ; ambil nilai maksimum hasilnya dengan `.max()`.

**C6.** Hitung first difference: `d1 = np.diff(y)`. Cetak mean dari `d1` (harus dekat dengan nilai tren rata-rata per hari).

- 💡 **HINT:** Hitung first difference dengan `np.diff(y)`, lalu rata-ratakan dengan `np.mean`.

**C7.** Hitung moving average window 7 (mingguan) via pandas: `ma7 = series.rolling(7).mean()`. Cetak nilai pada `ma7.iloc[50]`.

- 💡 **HINT:** Gunakan `series.rolling(window=7).mean()`, lalu ambil nilai pada indeks yang diminta dengan `.iloc`.

**C8.** Hitung Exponential Moving Average  $\alpha=0,1$  via pandas: `ema = series.ewm(alpha=0.1, adjust=False).mean()`. Cetak nilai pada `ema.iloc[-1]`.

- 💡 **HINT:** Gunakan `series.ewm(alpha=..., adjust=False).mean()`, lalu ambil nilai pada indeks terakhir dengan `.iloc[-1]`.

**C9.** Terapkan Box-Cox transform dengan `scipy.stats.boxcox` pada dataset referensi. Cetak nilai parameter lambda optimal.

- 💡 **HINT:** Pakai `scipy.stats.boxcox`; fungsi mengembalikan data transformasi dan parameter lambda optimal sebagai nilai kedua.

**C10.** Jalankan Augmented Dickey-Fuller test pada deret hasil differencing orde-1 (`np.diff(y)`). Cetak p-value hasilnya.

- 💡 **HINT:** Gunakan `statsmodels.tsa.stattools.adfuller` pada deret hasil differencing; p-value adalah elemen indeks ke-1 dari tuple hasilnya.

### Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

**C11 (Hard).** Implementasikan dekomposisi klasik aditif dari awal (tanpa `statsmodels`). Untuk dataset referensi dengan `periode=7`: (1) hitung tren  $T$  dengan centered moving average window 7, (2) hitung detrended  $= y - T$ , (3) hitung komponen musiman  $S$  dengan rata-rata detrended per posisi hari ( $\text{mod } 7$ ), (4) hitung residual  $R = y - T - S$ . Laporkan standar deviasi residual.

- 💡 **HINT:** Hitung tren  $T$  dengan centered moving average periode 7, detrended  $= y - T$ ; komponen musiman  $S$  = rata-rata detrended per posisi hari ( $\text{mod } 7$ ); residual  $= y - T - S$ , lalu hitung std-nya (abaikan NaN ujung).

**C12 (Hard).** Implementasikan Savitzky-Golay filter dari awal (tanpa `scipy.signal.savgol_filter`) untuk `window=11` dan orde polinomial  $p=2$ . Terapkan pada dataset referensi. Laporkan nilai filtered pada indeks 200.

- 💡 **HINT:** Untuk tiap pusat window, selesaikan least-squares polinomial orde  $p$  dengan `np.polyfit` pada `y[i-5:i+6]`, lalu ambil nilai polinomial di pusat. Di tepi, ambil nilai asli. Bandingkan dengan `scipy.signal.savgol_filter` untuk validasi.

**C13 (Hard).** Jalankan STL decomposition dengan statsmodels pada dataset referensi (periode=7, robust=True). Laporkan persentase varians yang dijelaskan komponen tren:  $\text{var}(\text{trend})/\text{var}(y) \times 100$ .

- 💡 **HINT:** Jalankan STL dari statsmodels.tsa.seasonal, ambil komponen .trend, lalu hitung rasio  $\text{var}(\text{trend}) / \text{var}(y) \times 100$ .

**C14 (Hard).** Implementasikan deteksi anomali berbasis EMA: (1) hitung EMA dengan  $\alpha=0,1$ , (2) hitung residual =  $y - \text{EMA}$ , (3) hitung sigma\_residual, (4) hitung jumlah titik dimana  $|\text{residual}| > 3 \cdot \text{sigma\_residual}$ . Sebelum hitung, injeksikan 5 anomali buatan pada indeks [50,100,150,200,300] dengan  $y_{\text{mod}} \pm 10$ . Laporkan jumlah anomali terdeteksi pada  $y_{\text{mod}}$ .

- 💡 **HINT:** Hitung EMA dengan `ewm(...).mean()`, residual =  $y_{\text{mod}} - \text{EMA}$ , lalu hitung jumlah titik dengan  $|\text{residual}|$  melebihi  $3 \times \text{std residual}$ .

**C15 (Hard).** Implementasikan pipeline preprocessing lengkap: (1) log transform, (2) Z-score scaling dengan fit hanya pada 80% data training pertama, (3) differencing orde 1, (4) ADF test pada hasil akhir. Laporkan nilai mean absolut dari hasil akhir pipeline (seharusnya sangat kecil karena differencing + standardisasi).

- 💡 **HINT:** Urutan: (1) log transform dengan `np.log`; (2) fit scaler pada 80% data awal lalu transform semua; (3) differencing dengan `np.diff`; (4) laporkan nilai absolut dari mean hasil differencing.

## DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. *IEEE Transactions on Knowledge and Data Engineering*, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972.

<https://doi.org/10.3390/app12030972>

Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>